

AccessFlow API Developer Documentation Sample

OpenAPI 3.1 Contract, Developer Guidance, Examples, and Validation

Michael K. Levesque

September 2026

ORIGINAL API DOCUMENTATION PORTFOLIO

A compact, fictional identity-governance API documented as a Git-friendly OpenAPI 3.1 contract with security scopes, reusable schemas, examples, lifecycle rules, error handling, and validation.

PORTFOLIO CONTENTS

OpenAPI 3.1 Specification | Developer Guide | JSON Examples | Validation Script | Changelog

Project overview

AccessFlow is a fictional REST API for discovering requestable entitlements, submitting application-access requests, recording approval decisions, and tracking request state. The sample is designed to show API documentation judgment rather than to imitate a production platform.

The source-of-truth contract is an OpenAPI 3.1 YAML file. Human-facing guidance explains the same contract from the developer's perspective: how to authenticate, form requests, retry safely, interpret lifecycle states, diagnose failures, and correlate API errors with service logs.

Portfolio objective

- Demonstrate OpenAPI 3.1 source authoring using reusable parameters, headers, schemas, responses, and examples.
- Translate identity-governance behavior into a small resource model that a developer can understand quickly.
- Document authentication and authorization explicitly through OAuth 2.0 scopes.
- Show operational concerns - idempotency, request tracing, errors, state conflicts, pagination, and validation - not only happy-path endpoint descriptions.
- Keep the project suitable for Git review, automated validation, and later documentation generation.

Audience and use cases

Reader	What they need from the documentation
Application developer	Authentication, endpoint contract, request/response examples, retry behavior, and error handling.
Identity / IAM engineer	Entitlement model, subject identifiers, approval behavior, access state, and authorization scopes.
Reviewer / architect	Resource boundaries, consistency, security assumptions, reuse, versioning, and lifecycle semantics.
Support engineer	Correlation IDs, stable problem codes, expected states, and information needed to investigate failures.
PORTFOLIO BOUNDARY All server names use the reserved .test domain. Identities, entitlement names, examples, and business rules are fictional. No employer, customer, or proprietary data is used.	

Contract design and security

The API uses a small set of predictable resources and operations. Read and write permissions are separated so a service can receive only the capability it needs.

OAuth 2.0 scopes

Scope	Purpose
access-requests.read	Discover entitlements and read access-request state.
access-requests.write	Create requests and cancel requests that are still pending.
access-decisions.write	Approve or deny requests assigned to the caller.

The OpenAPI security scheme uses the OAuth 2.0 client-credentials flow for service-to-service access. Authentication proves the caller possesses a valid token; operation-level scopes determine whether that caller is authorized to perform a particular action.

Cross-cutting request conventions

Convention	Behavior
Authorization	Bearer access token. The required scope is documented per operation.
Content type	JSON requests use application/json. Structured errors use application/problem+json.
X-Correlation-ID	Optional caller-supplied UUID. The service returns the effective value for log correlation.
Idempotency-Key	Required on write operations where safe retry matters. Equivalent retries return the original result.
Dates and times	Dates use ISO 8601 YYYY-MM-DD. Timestamps use RFC 3339 / ISO 8601 UTC date-time values.
Pagination	List operations use an opaque cursor rather than exposing database offsets.

Why idempotency is documented

Clients often cannot tell whether a timed-out POST reached the service. Repeating the request without protection can create duplicates. AccessFlow requires an Idempotency-Key on request creation and decision submission so a client can retry the same logical operation safely. Reusing the same key with a different payload is treated as a conflict.

SECURITY ASSUMPTION

This portfolio documents the API boundary only. A production implementation would also define token lifetime, client registration, secret rotation, audit retention, rate limits, transport requirements, and authorization enforcement outside the OpenAPI contract.

Quickstart and request lifecycle

1. Obtain a token

```
curl -X POST 'https://auth.example.test/oauth2/token' \  
-u "$CLIENT_ID:$CLIENT_SECRET" \  
-H 'Content-Type: application/x-www-form-urlencoded' \  
--data-urlencode 'grant_type=client_credentials' \  
--data-urlencode 'scope=access-requests.write access-requests.read'
```

2. Submit an access request

```
curl -X POST 'https://api.example.test/accessflow/v1/access-requests' \  
-H "Authorization: Bearer $ACCESS_TOKEN" \  
-H 'Content-Type: application/json' \  
-H 'Idempotency-Key: 9bf6c87e-20ab-4f98-af18-fd8375832f6c' \  
--data '{  
  "subjectId": "usr-10452",  
  "entitlementId": "ent-reporting-read",  
  "justification": "Need read-only reporting access for quarter-end reconciliation.",  
  "requestedUntil": "2026-12-31"  
}'
```

Expected result: HTTP 201 Created, a Location header containing the canonical request URI, an X-Correlation-ID response header, and a representation whose status is pending.

Lifecycle rules

Current state	Allowed next states	Reason
pending	approved, denied, cancelled, expired	The request is still awaiting completion.
approved	expired	Granted access may later reach its end date.
denied	none	Decision is terminal for this request.
cancelled	none	Requester ended the request before completion.
expired	none	Request or granted access is no longer active.

CONFLICT VS VALIDATION

A well-formed request can still be invalid for the resource's current state. AccessFlow uses HTTP 409 for state conflicts (for example, approving a request that is no longer awaiting that approver) and HTTP 422 for field or business validation failures.

Endpoint reference

Method	Path	Purpose	Scope
GET	/v1/entitlements	List requestable entitlements.	access-requests.read
POST	/v1/access-requests	Create a new access request.	access-requests.write
GET	/v1/access-requests/{requestId}	Return current request state and decision history.	access-requests.read
POST	/v1/access-requests/{requestId}/decisions	Record approval or denial.	access-decisions.write
POST	/v1/access-requests/{requestId}/cancel	Cancel a request that is still pending.	access-requests.write

POST /v1/access-requests

Creates one request for one subject and one entitlement. The caller supplies the business justification and may request an expiration date. The service assigns the request identifier, submission timestamp, submitter, and initial lifecycle state.

Request body

Field	Required	Type	Notes
subjectId	Yes	string	Stable person identifier; sample pattern usr-#####.
entitlementId	Yes	string	Identifier returned by entitlement discovery.
justification	Yes	string	20-1000 characters; explains the business need.
requestedUntil	No	date or null	Requested end date; null means no requester-specified end date.

Success response

```
{
  "requestId": "arq-01K5JQ8YDX2X64QAY4R4A9T7DG",
  "subjectId": "usr-10452",
  "entitlementId": "ent-reporting-read",
  "status": "pending",
  "submittedAt": "2026-09-22T18:42:31Z",
  "submittedBy": "usr-10452",
  "decisions": []
}
```

Selected responses

HTTP	Meaning
201	Request created successfully.
409	An active duplicate exists or an idempotency key was reused with different content.
422	Payload is valid JSON but fails documented field or business validation.

Errors, diagnostics, and supportability

A useful API reference must explain failure behavior as clearly as success behavior. AccessFlow returns structured problem details using application/problem+json. The shape remains consistent across endpoints so client code can handle failures predictably.

Problem response

```
{
  "type": "https://api.example.test/problems/validation-error",
  "title": "Validation failed",
  "status": 422,
  "detail": "One or more fields failed validation.",
  "code": "validation_error",
  "correlationId": "3ae9a6d3-ae6-4f55-b70e-6401cd1d98f6",
  "errors": [
    {
      "field": "justification",
      "message": "must contain at least 20 characters"
    }
  ]
}
```

Error semantics

HTTP	Use in this API	Client action
400	Malformed request syntax or invalid parameter representation.	Correct the request before retrying.
401	Token missing, invalid, or expired.	Obtain a valid token and retry.
403	Caller authenticated but lacks required scope/resource access.	Do not retry unchanged; correct authorization.
404	Resource absent or not visible to the caller.	Verify identifier and caller visibility.
409	Current resource state conflicts with requested action.	Refresh state before deciding whether to retry.
422	Documented validation rule failed.	Correct the reported field/business issue.
500	Unexpected service failure.	Retry according to client policy; retain correlation ID.

Correlation workflow

- Client may send X-Correlation-ID as a UUID. If omitted, the service generates one.
- The service returns the effective value in the X-Correlation-ID response header.
- Problem responses repeat the value in the response body so logs and user-visible errors can be tied together.
- Support guidance should ask for correlation ID, timestamp, operation, HTTP status, and non-secret request context - never access tokens or client secrets.

Specification quality and validation

The portfolio keeps the API contract in source form so it can participate in a docs-as-code workflow. The YAML file is readable in review, diffable in Git, and structured for future linting or documentation generation.

OpenAPI 3.1 features demonstrated

- OpenAPI 3.1.0 with JSON Schema Draft 2020-12 semantics.
- Reusable component schemas, parameters, response objects, headers, and examples.
- Explicit nullability using JSON Schema type arrays instead of the older nullable keyword.
- Operation-specific OAuth scope requirements.
- Schema constraints including patterns, enumerations, minimum lengths, formats, and additionalProperties: false.
- Reusable structured error contracts and documented media types.

Included validation script

The package includes `scripts/validate_examples.py`. It is intentionally lightweight and does not pretend to replace a full OpenAPI linter. It verifies the items most likely to become inconsistent while editing this sample:

- the YAML parses and declares OpenAPI 3.1;
- every documented operation has a summary, unique operationId, and responses;
- internal \$ref targets resolve;
- the supplied request, success-response, and error JSON files validate against their documented JSON Schemas.

```
$ python scripts/validate_examples.py
PASS: OpenAPI structure, internal refs, operation IDs, and example payloads validated.
```

Repository structure

```
AccessFlow_OpenAPI_Portfolio/
├── README.md
├── openapi.yaml
├── CHANGELOG.md
├── docs/
│   ├── AccessFlow_API_Developer_Guide.pdf
│   └── AccessFlow_API_Developer_Guide.docx
├── examples/
│   ├── create-access-request.json
│   ├── access-request-approved.json
│   └── problem-validation.json
├── scripts/
│   └── validate_examples.py
```

Documentation decisions and portfolio status

Design decisions

Decision	Documentation rationale
Resource-oriented paths	Paths describe business resources and subordinate actions rather than implementation internals.
Stable operation IDs	Names such as <code>createAccessRequest</code> and <code>createDecision</code> support SDK generation and durable cross-reference.
Problem-details error shape	One reusable contract reduces special-case client handling and improves diagnostics.
Explicit state conflicts	409 communicates that syntax may be correct while the current resource state prevents the action.
Opaque pagination cursor	Avoids coupling the public contract to storage offsets or query implementation.
Reserved <code>.test</code> domains	Makes it unmistakable that the sample endpoints are documentation-only and non-production.

What I authored

I designed the fictional workflow, information model, resource paths, authentication scopes, schemas, request and response contracts, examples, error model, lifecycle guidance, validation approach, README, and this developer guide. The work is intended to demonstrate API-documentation capability rather than production software implementation.

What this sample does not claim

- The API is not deployed and the server URLs are not callable.
- The validation script is a portfolio-level consistency check, not a substitute for a production OpenAPI linter, contract-test suite, or CI pipeline.
- The sample does not claim experience operating this exact API in production.
- Security controls beyond the documented API boundary would require implementation-specific design and verification.

PORTFOLIO STATUS

Original professional-evaluation sample by Michael K. Levesque. Product and protocol names are used only to demonstrate documentation practices. No confidential third-party information is included.

OPENAPI 3.1 • API REFERENCE • JSON SCHEMA • DOCS AS CODE • VALIDATION